

# Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

betreut durch Gutachter

Der Autor (Firma)

*Email: der.autor@example.com*

Leibniz-Fachhochschule

*Matr. Nr: 5*

14.08.2026

2026-08-14

Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick  
betreut durch Gutachter

Der Autor (Firma)  
*Email: der.autor@example.com*  
Leibniz-Fachhochschule  
Matr. Nr: 5  
14.08.2026

|      |                               |
|------|-------------------------------|
| CPU  | Central Processing Unit       |
| CbCC | Cache-basierte Covert-Channel |
| OoO  | Out-of-Order                  |
| TEA  | Transient Execution Angriff   |
| VM   | Virtuelle Maschine            |

- ➊ Problemstellung und Ansatz
- ➋ Umsetzung und Ergebnisse
- ➌ Ausblick
- ➍ Literatur

2026-08-14

Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

└ Inhaltsverzeichnis

- Inhaltsverzeichnis
- ➊ Problemstellung und Ansatz
- ➋ Umsetzung und Ergebnisse
- ➌ Ausblick
- ➍ Literatur

- TEAs sind durch Covert-Channel begrenzt  
→ CPU-Rollback erfordert zwangsläufig Übertragung
- CbCC häufig verwendet [5, S. 23] → hoher Durchsatz mit geringer Komplexität
- Abhängigkeit der Laufzeit von Übertragungen von Datenmenge und Durchsatz  
→ Durchsatz vorrangig durch Covert-Channel bestimmt

2026-08-14

## Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

- └ Problemstellung und Ansatz

- └ Problem

- TEAs durch Übertragung begrenzt (da Rollback praktisch einen Angriff definiert)- CbCC meist verwendet -  $\propto$  Durchsatz, Komplexität, meist hardwareunabhängig, viel Literatur...- Laufzeit ergibt sich aus Größe übertragener Daten und Durchsatz- Menge Daten abhängig von konkretem Angriff -  $\propto$  Durchsatz abhängig von Covert Channel -  $\propto$  allgemeingültig (besonders Encodierung, Rest begrenzt durch technische Grenzen)Fragestellung: Grenze von CbCC bei TEAs was die Laufzeit (damit Durchsatz) angeht

Problem

- TEAs sind durch Covert-Channel begrenzt  
→ CPU-Rollback erfordert zwangsläufig Übertragung
- CbCC häufig verwendet [5, S. 23] → hoher Durchsatz mit geringer Komplexität
- Abhängigkeit der Laufzeit von Übertragungen von Datenmenge und Durchsatz  
→ Durchsatz vorrangig durch Covert-Channel bestimmt

# Problem

- TEAs sind durch Covert-Channel begrenzt  
→ CPU-Rollback erfordert zwangsläufig Übertragung
- CbCC häufig verwendet [5, S. 23] → hoher Durchsatz mit geringer Komplexität
- Abhängigkeit der Laufzeit von Übertragungen von Datenmenge und Durchsatz  
→ Durchsatz vorrangig durch Covert-Channel bestimmt

## Fragestellung

Wo liegt die Grenze für CbCC im Kontext von TEAs hinsichtlich der Laufzeit und damit hinsichtlich des Durchsatzes?

2026-08-14

## Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

- └ Problemstellung und Ansatz

### └ Problem

- TEAs durch Übertragung begrenzt (da Rollback praktisch einen Angriff definiert)- CbCC meist verwendet -> Durchsatz, Komplexität, meist hardwareunabhängig, viel Literatur...- Laufzeit ergibt sich aus Größe übertragener Daten und Durchsatz- Menge Daten abhängig von konkretem Angriff -> Durchsatz abhängig von Covert Channel -> allgemeingültig (besonders Encodierung, Rest begrenzt durch technische Grenzen)Fragestellung: Grenze von CbCC bei TEAs was die Laufzeit (damit Durchsatz) angeht

Problem

- TEAs sind durch Covert-Channel begrenzt  
→ CPU-Rollback erfordert zwangsläufig Übertragung
- CbCC häufig verwendet [5, S. 23] → hoher Durchsatz mit geringer Komplexität
- Abhängigkeit der Laufzeit von Übertragungen von Datenmenge und Durchsatz  
→ Durchsatz vorrangig durch Covert-Channel bestimmt

Fragestellung

Wo liegt die Grenze für CbCC im Kontext von TEAs hinsichtlich der Laufzeit und damit hinsichtlich des Durchsatzes?

- häufig Array-Index Encodierung für CbCCs
- bitweise Encodierung bisher nicht implementiert oder getestet  
→ nur Kombination aus beiden [9]

2026-08-14

## Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

- └ Problemstellung und Ansatz

- └ Ansatz

- bisherige Angriffe nutzen meist Array-Index Enc. -> Ausnahme l1tf reloaded -> Encodierung 1 Byte als 2x 16 Adressen -> keine reine bitweise Enc. -> Grenzen nicht getestet- reine bitweise Encodierung nicht betrachtet- Implementierung beider Implementierungen, um grundsätzliche Umsetzbarkeit zu testen- Erster Vergleich bitweise und Array-Index Encodierung- Laufzeitbetrachtung mit Entwicklung einer Funktion- ermitteln einer Grenze der Laufzeit / vom Durchsatz von CbCC

- häufig Array-Index Encodierung für CbCCs
- bitweise Encodierung bisher nicht implementiert oder getestet  
→ nur Kombination aus beiden [9]

- häufig Array-Index Encodierung für CbCCs
- bitweise Encodierung bisher nicht implementiert oder getestet  
→ nur Kombination aus beiden [9]

## Ziel 1

Implementieren von Bitweiser Encodierung während Transient Execution, um die Umsetzbarkeit verschiedener Encodierungen zu zeigen. Zusätzlich können Aspekte wie Genauigkeit und Laufzeit betrachtet werden.

2026-08-14

## Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

- └ Problemstellung und Ansatz

### └ Ansatz

- bisherige Angriffe nutzen meist Array-Index Enc. -i Ausnahme l1tf reloaded -i Encodierung 1 Byte als 2x 16 Adressen -i keine reine bitweise Enc. -i Grenzen nicht getestet- reine bitweise Encodierung nicht betrachtet- Implementierung beider Implementierungen, um grundsätzliche Umsetzbarkeit zu testen- Erster Vergleich bitweise und Array-Index Encodierung- Laufzeitbetrachtung mit Entwicklung einer Funktion- ermitteln einer Grenze der Laufzeit / vom Durchsatz von CbCC

Ansatz

- häufig Array-Index Encodierung für CbCCs
- bitweise Encodierung bisher nicht implementiert oder getestet  
→ nur Kombination aus beiden [9]

### Ziel 1

Implementieren von Bitweiser Encodierung während Transient Execution, um die Umsetzbarkeit verschiedener Encodierungen zu zeigen. Zusätzlich können Aspekte wie Genauigkeit und Laufzeit betrachtet werden.

- häufig Array-Index Encodierung für CbCCs
- bitweise Encodierung bisher nicht implementiert oder getestet  
→ nur Kombination aus beiden [9]

## Ziel 1

Implementieren von Bitweiser Encodierung während Transient Execution, um die Umsetzbarkeit verschiedener Encodierungen zu zeigen. Zusätzlich können Aspekte wie Genauigkeit und Laufzeit betrachtet werden.

## Ziel 2

Modellierung der Laufzeit einer Übertragung sowie des Durchsatzes. Ermittlung einer Grenze für CbCCs während Transient Execution.

2026-08-14

## Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

### Problemstellung und Ansatz

### Ansatz

- bisherige Angriffe nutzen meist Array-Index Enc. -i Ausnahme l1tf reloaded -i Encodierung 1 Byte als 2x 16 Adressen -i keine reine bitweise Enc. -i Grenzen nicht getestet- reine bitweise Encodierung nicht betrachtet- Implementierung beider Implementierungen, um grundsätzliche Umsetzbarkeit zu testen- Erster Vergleich bitweise und Array-Index Encodierung- Laufzeitbetrachtung mit Entwicklung einer Funktion- ermitteln einer Grenze der Laufzeit / vom Durchsatz von CbCC

Ansatz

- häufig Array-Index Encodierung für CbCCs
- bitweise Encodierung bisher nicht implementiert oder getestet  
→ nur Kombination aus beiden [9]

#### Ziel 1

Implementieren von Bitweiser Encodierung während Transient Execution, um die Umsetzbarkeit verschiedener Encodierungen zu zeigen. Zusätzlich können Aspekte wie Genauigkeit und Laufzeit betrachtet werden.

#### Ziel 2

Modellierung der Laufzeit einer Übertragung sowie des Durchsatzes. Ermittlung einer Grenze für CbCCs während Transient Execution.



# Phasen des Angriffs

- wegen grundsätzlicher Funktionsweise bestehen TEAs aus 3 Phasen
- bei konkretem Angriff wiederholen dieser Phasen → stückweises übertragen von Secret
- Optimieren dieser Phasen für kürzere Laufzeiten

2026-08-14

Die Grenzen von cachebasierten Covert-Channels im Kontext von  
Transient-Execution: ein technischer Überblick  
└ Problemstellung und Ansatz

└ Phasen des Angriffs

Erklärung Unterschied von Side und Covert Channel (Encode bewusst, unbewusst)

Phasen des Angriffs

- wegen grundsätzlicher Funktionsweise bestehen TEAs aus 3 Phasen
- bei konkretem Angriff wiederholen dieser Phasen → stückweises übertragen von Secret
- Optimieren dieser Phasen für kürzere Laufzeiten

# Phasen des Angriffs

- wegen grundsätzlicher Funktionsweise bestehen TEAs aus 3 Phasen
- bei konkretem Angriff wiederholen dieser Phasen → stückweises übertragen von Secret
- Optimieren dieser Phasen für kürzere Laufzeiten

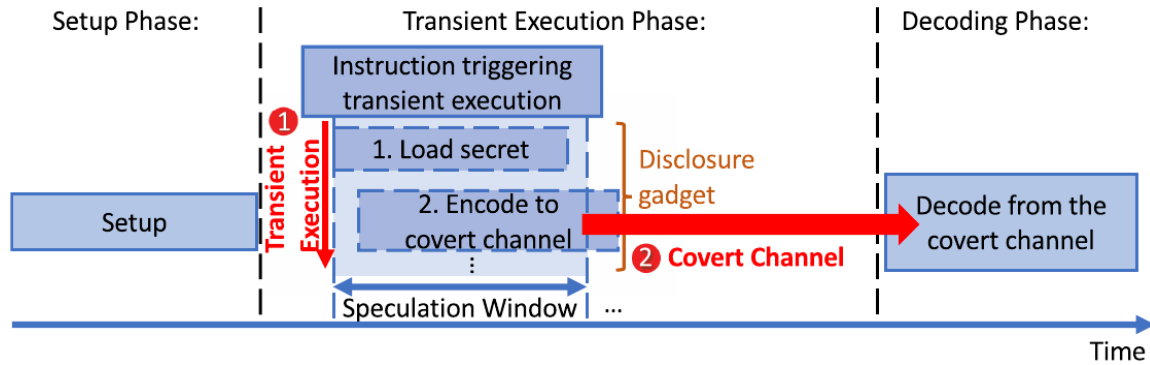
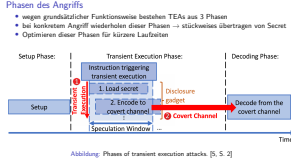


Abbildung: Phases of transient execution attacks. [5, S. 2]

2026-08-14

## Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

- └ Problemstellung und Ansatz
- └ Phasen des Angriffs



Erklärung Unterschied von Side und Covert Channel (Encode bewusst, unbewusst)

Beispiel: Encodierung des Werts 7 ( $1110_2$ ) in vier Bit (Encodierungen ähnlich zu [3])

2026-08-14

Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

└ Umsetzung und Ergebnisse

└ CbCC Laufzeit

CbCC Laufzeit

Beispiel: Encodierung des Werts 7 ( $1110_2$ ) in vier Bit (Encodierungen ähnlich zu [3])

- Unterscheidung bitweise und Array-Index Encodierung -; Ursprünglich erstmals unterschieden in zitierter Quelle- Andere Benennung für Encodierung: Bitweise -; Direct-mapped bits; Array-Index -; Index values- Encodierung von 0 bei Index values als seperater Zustand- binäre Darstellung least significant bit first (von links nach rechts)!!!!- Erklären wie die Adressen im folgenden dargestellt (gelbe & grüne Kästchen)- Herleitung Formeln erklären (für reine bitweise und Array-Index Encodierung)- Anzahl an Adressen  $n_a$  wird  $n_b$  mal ausgeführt

Beispiel: Encodierung des Werts 7 ( $1110_2$ ) in vier Bit (Encodierungen ähnlich zu [3])

**Bitweise:**



2026-08-14

Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

└ Umsetzung und Ergebnisse

└ CbCC Laufzeit

- bitweise Encodierung sehr Intuitiv- praktisch encodieren durch Bitmask und load nach Branch-Herleitung Formeln erklären (für reine bitweise und Array-Index Encodierung)- Anzahl an Adres-sen  $n_a$  wird  $n_b$  mal ausgeführt

CbCC Laufzeit

Beispiel: Encodierung des Werts 7 ( $1110_2$ ) in vier Bit (Encodierungen ähnlich zu [3])

Bitweise:

■ ■ ■ ■

Beispiel: Encodierung des Werts 7 ( $1110_2$ ) in vier Bit (Encodierungen ähnlich zu [3])

**Bitweise:**



**Array-Index:**



2026-08-14

Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick  
└ Umsetzung und Ergebnisse

└ CbCC Laufzeit



- Array Index Intuitiv- Auch Bitmask (nur betrachtete Bits übertragen)- Wert als Offset bei einer Load Instruktion— bitweise nur so viele Adressen nötig, wie encodiert- Array-Index exponentielle Anzahl an Adressen -i flushen und messen- Herleitung Formeln erklären (für reine bitweise und Array-Index Encodierung)- Anzahl an Adressen n\_a wird n\_b mal ausgeführt

Beispiel: Encodierung des Werts 7 (1110<sub>2</sub>) in vier Bit (Encodierungen ähnlich zu [3])

Bitweise:



Array-Index:



Kombination mit Parametern  $n_b = 2$  und  $n_a = 2$ :



2026-08-14

Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick  
└ Umsetzung und Ergebnisse

└ CbCC Laufzeit



- Encodierung von  $n_a$  Bits in  $n_b$  Gruppen— in diesem Falle immer 2 Bit als Array Index Encodierung- hohe Komplexität (Bitmaske mit  $n_a$  Bit -> schieben und jedes Mal Index encodieren)- Herleitung Formeln erklären (für reine bitweise und Array-Index Encodierung)- Anzahl an Adressen  $n_a$  wird  $n_b$  mal ausgeführt

Beispiel: Encodierung des Werts 7 (1110<sub>2</sub>) in vier Bit (Encodierungen ähnlich zu [3])

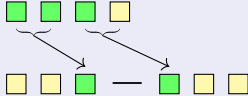
Bitweise:



Array-Index:



Kombination mit Parametern  $n_b = 2$  und  $n_a = 2$ :



2026-08-14

Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

└ Umsetzung und Ergebnisse

└ CbCC Laufzeit



- Encodierung von  $n_a$  Bits in  $n_b$  Gruppen— in diesem Falle immer 2 Bit als Array Index Encodierung- hohe Komplexität (Bitmaske mit  $n_a$  Bit -> schieben und jedes Mal Index encodieren)- Herleitung Formeln erklären (für reine bitweise und Array-Index Encodierung)- Anzahl an Adressen  $n_a$  wird  $n_b$  mal ausgeführt

Beispiel: Encodierung des Werts 7 (1110<sub>2</sub>) in vier Bit (Encodierungen ähnlich zu [3])

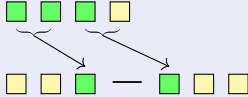
Bitweise:



Array-Index:



Kombination mit Parametern  $n_b = 2$  und  $n_a = 2$ :



$$N_P = n_b * (2^{n_a} - 1) \qquad N_R = n_b * 2^{n_a - 1}$$

- Array-Index encodiert 0 als keine gecachte Adresse → vereinfacht die Formel
- vorbereitete Adressen und gemessene Adressen wird unterschieden → dafür keine Betrachtung des Zeitunterschieds von gecachten und ungecachten Adressen
- obere Grenze für Durchsatz, da bei entsprechend verwendeten Parametern die Laufzeit unterschätzt wird

2026-08-14

Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick  
└ Umsetzung und Ergebnisse

└ CbCC Laufzeit

- Herleitung Formeln erklären (für reine bitweise und Array-Index Encodierung)- Anzahl an Adres-  
sen n\_a wird n\_b mal ausgeführt

CbCC Laufzeit

Beispiel: Encodierung des Werts 7 (1110<sub>2</sub>) in vier Bit (Encodierungen ähnlich zu [3])

Bitweise:

Array-Index:

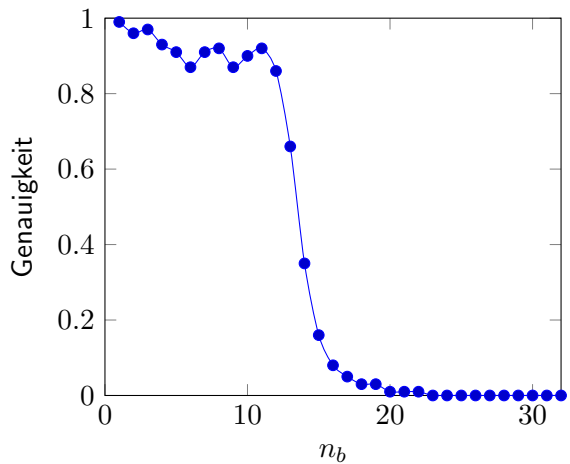
Kombination mit Parametern  $n_b = 2$  und  $n_a = 2$ :

$$N_P = n_b * (2^{n_a} - 1) \qquad N_R = n_b * 2^{n_a - 1}$$

- Array-Index encodiert 0 als keine gecachte Adresse → vereinfacht die Formel
- vorbereitete Adressen und gemessene Adressen wird unterschieden → dafür keine Betrachtung des Zeitunterschieds von gecachten und ungecachten Adressen
- obere Grenze für Durchsatz, da bei entsprechend verwendeten Parametern die Laufzeit unterschätzt wird



# Genauigkeit und Parameter

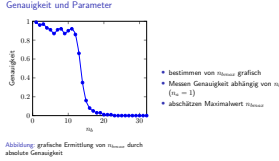


- bestimmen von  $n_{bmax}$  grafisch
- Messen Genauigkeit abhängig von  $n_b$  ( $n_a = 1$ )
- abschätzen Maximalwert  $n_{bmax}$

Abbildung: grafische Ermittlung von  $n_{bmax}$  durch absolute Genauigkeit

2026-08-14 Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

- └ Umsetzung und Ergebnisse
- └ Genauigkeit und Parameter



# Laufzeit der kombinierten Encodierung

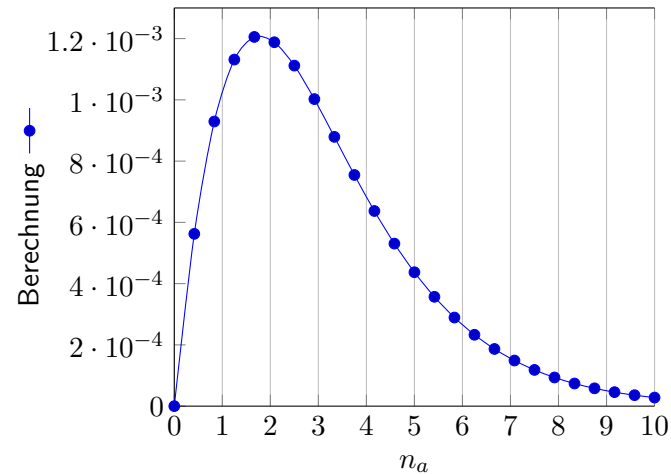


Abbildung:  $D(n_a)$  mit  $N_T = 20$ ,  $T_T = 109$ ,  $L_E = 569$ ,  $T_P = 144$ ,  $T_R = 410$  und  $n_{bmax} = 10$

2026-08-14

Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

└ Umsetzung und Ergebnisse

└ Laufzeit der kombinierten Encodierung

- etwa Faktor 20 geringer als berechnet- keine Betrachtung der Genauigkeit- genauere Einschätzung des Durchsatzes möglich

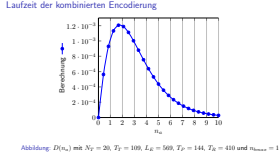


Abbildung:  $D(n_a)$  mit  $N_T = 20$ ,  $T_T = 109$ ,  $L_E = 569$ ,  $T_P = 144$ ,  $T_R = 410$  und  $n_{bmax} = 10$

# Laufzeit der kombinierten Encodierung

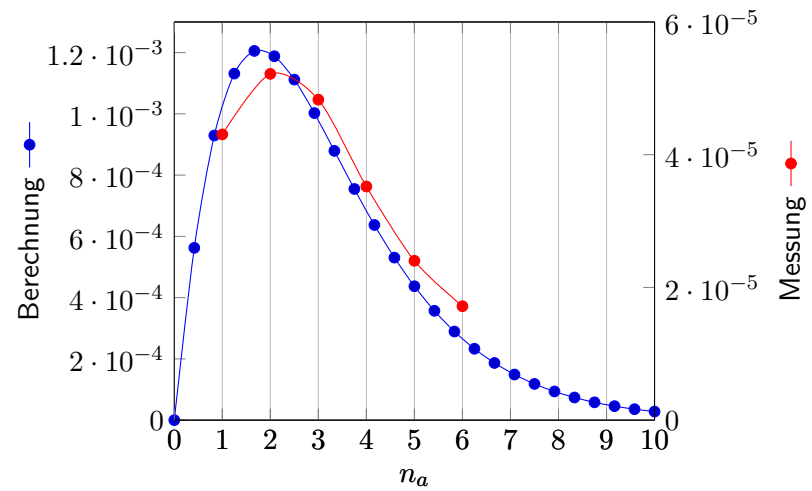


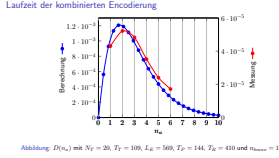
Abbildung:  $D(n_a)$  mit  $N_T = 20$ ,  $T_T = 109$ ,  $L_E = 569$ ,  $T_P = 144$ ,  $T_R = 410$  und  $n_{bmax} = 10$

2026-08-14

Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

- └ Umsetzung und Ergebnisse
- └ Laufzeit der kombinierten Encodierung

- etwa Faktor 20 geringer als berechnet- keine Betrachtung der Genauigkeit- genauere Einschätzung des Durchsatzes möglich



# Laufzeit der kombinierten Encodierung

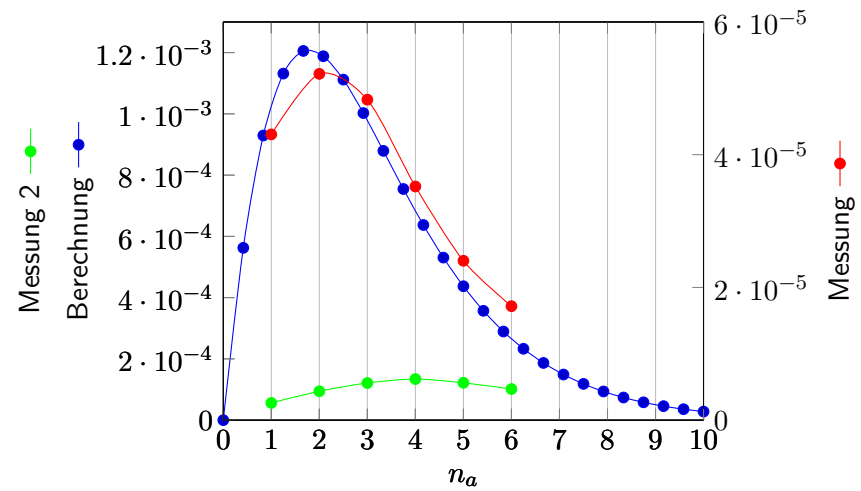


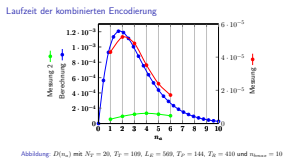
Abbildung:  $D(n_a)$  mit  $N_T = 20$ ,  $T_T = 109$ ,  $L_E = 569$ ,  $T_P = 144$ ,  $T_R = 410$  und  $n_{bmax} = 10$

2026-08-14

## Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

- └ Umsetzung und Ergebnisse
- └ Laufzeit der kombinierten Encodierung

- etwa Faktor 20 geringer als berechnet- keine Betrachtung der Genauigkeit- genauere Einschätzung des Durchsatzes möglich



# Genauigkeit & Fehlerkorrekturen

- nur Betrachtung von reiner Laufzeit → Annahme keine Fehler
- praktisch betrachtet Fehlerkorrektur nötig → stark abhängig von Encodierung

2026-08-14

Die Grenzen von cachebasierten Covert-Channels im Kontext von  
Transient-Execution: ein technischer Überblick  
└─Ausblick

└─Genauigkeit & Fehlerkorrekturen

- in dieser Arbeit nur Betrachtung der Laufzeit- für realistischen Durchsatz müsste Fehlerwahrscheinlichkeit und Fehlerkorrektur betrachtet werden- Betrachtung Fehlerkorrektur abhängig von Hardware -¿ wahrscheinlich komplex (Anzahl ausführbarer Instruktionen)- einfachste naiveF-Fehlerkorrektur durch wiederholtes Ausführen -¿ Häufigkeit analysieren

Genauigkeit & Fehlerkorrekturen

- nur Betrachtung von reiner Laufzeit → Annahme keine Fehler
- praktisch betrachtet Fehlerkorrektur nötig → stark abhängig von Encodierung

- nur Betrachtung von reiner Laufzeit → Annahme keine Fehler
- praktisch betrachtet Fehlerkorrektur nötig → stark abhängig von Encodierung

## Bitweise Encodierung

- theoretisch unabhängige Datenpunkten  
→ jeweils Fehlerkorrektur
- da unabhängige Daten sind komplexere Verfahren möglich  
→ fraglich, ob praktisch effizient (während Transient Execution)

2026-08-14

## Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

- └ Ausblick

### └ Genauigkeit & Fehlerkorrekturen

- Vorteil theoretisch unabhängiger Datenpunkte- jeweils naive Fehlerkorrektur möglich- komplexer wie durch Parität oder 2x mal übertragen möglich- mehr Instruktionen für Fehlerkorrektur durch Transient Window begrenzt -> nur sehr eingeschränkt möglich -> Nutzen fraglich

- nur Betrachtung von reiner Laufzeit → Annahme keine Fehler
- praktisch betrachtet Fehlerkorrektur nötig → stark abhängig von Encodierung

#### Bitweise Encodierung

- theoretisch unabhängige Datenpunkten  
→ jeweils Fehlerkorrektur
- da unabhängige Daten sind komplexere Verfahren möglich  
→ fraglich, ob praktisch effizient (während Transient Execution)

- nur Betrachtung von reiner Laufzeit → Annahme keine Fehler
- praktisch betrachtet Fehlerkorrektur nötig → stark abhängig von Encodierung

## Bitweise Encodierung

- theoretisch unabhängige Datenpunkten  
→ jeweils Fehlerkorrektur
- da unabhängige Daten sind komplexere Verfahren möglich  
→ fraglich, ob praktisch effizient (während Transient Execution)

## Array-Index Encodierung

- Betrachtung der häufigsten Werte möglich  
→ 0 als Index encodieren  
→ alle Adressen messen
- weniger übertragene unabhängige Informationen  
→ weniger Möglichkeiten als bitweise

2026-08-14

## Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

└ Ausblick

### └ Genauigkeit & Fehlerkorrekturen

- naive Fehlerkorrektur -  $\hat{n}$  sowohl lesen bis Adresse gecached oder alle lesen möglich -  $\hat{n}$  im Idealfall alle Adressen messen -  $\hat{n}$  höhere Laufzeitkomplexität ( $2^{\hat{n}}$  anstatt  $2^{(n-1)}$ ) - 0 sollte als separater Zustand encodiert werden (könnte statistisch wahrscheinlich auch modelliert werden, wenn alle Adressen messen) - Änderung der Formeln nötig - generell weniger unabhängige Informationen - Fehlerkorrektur noch eingeschränkter (nicht nur Transient Window, auch Encodierung)

- nur Betrachtung von reiner Laufzeit → Annahme keine Fehler
- praktisch betrachtet Fehlerkorrektur nötig → stark abhängig von Encodierung

#### Bitweise Encodierung

- theoretisch unabhängige Datenpunkten  
→ jeweils Fehlerkorrektur
- da unabhängige Daten sind komplexere Verfahren möglich  
→ fraglich, ob praktisch effizient (während Transient Execution)

#### Array-Index Encodierung

- Betrachtung der häufigsten Werte möglich  
→ 0 als Index encodieren  
→ alle Adressen messen
- weniger übertragene unabhängige Informationen  
→ weniger Möglichkeiten als bitweise

# Literatur I

[1] Jo Van Bulck, Daniel Genkin, Baris Kasikci, KU Leuven, Mark Silberstein, Thomas F Wenisch und Yuval Yarom. “Foreshadow: Extracting the Keys to the Intel SGX Kingdom with Transient Out-of-Order Execution”. In: *27th USENIX Security Symposium (USENIX Security 18)* (2018).

[2] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz und Yuval Yarom. “Spectre Attacks: Exploiting Speculative Execution”. In: *2019 IEEE Symposium on Security and Privacy (SP)*. 2019 IEEE Symposium on Security and Privacy (SP). San Francisco, CA, USA: IEEE, Mai 2019, S. 1–19. ISBN: 978-1-5386-6660-9. DOI: 10.1109/SP.2019.00002.

[3] Ross Mcilroy, Jaroslav Sevcik, Tobias Tebbi, Ben L. Titzer und Toon Verwaest. *Spectre is here to stay: An analysis of side-channels and speculative execution*. 14. Feb. 2019. DOI: 10.48550/arXiv.1902.05178. arXiv: 1902.05178[cs].

2026-08-14

## Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

└ Literatur

└ Literatur

Literatur I

[1] Jo Van Bulck, Daniel Genkin, Baris Kasikci, KU Leuven, Mark Silberstein, Thomas F Wenisch und Yuval Yarom. “Foreshadow: Extracting the Keys to the Intel SGX Kingdom with Transient Out-of-Order Execution”. In: *27th USENIX Security Symposium (USENIX Security 18)* (2018).

[2] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz und Yuval Yarom. “Spectre Attacks: Exploiting Speculative Execution”. In: *2019 IEEE Symposium on Security and Privacy (SP)*. 2019 IEEE Symposium on Security and Privacy (SP). San Francisco, CA, USA: IEEE, Mai 2019, S. 1–19. ISBN: 978-1-5386-6660-9. DOI: 10.1109/SP.2019.00002.

[3] Ross Mcilroy, Jaroslav Sevcik, Tobias Tebbi, Ben L. Titzer und Toon Verwaest. *Spectre is here to stay: An analysis of side-channels and speculative execution*. 14. Feb. 2019. DOI: 10.48550/arXiv.1902.05178. arXiv: 1902.05178[cs].



# Literatur II

[4] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Jann Horn, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom, Mike Hamburg und Raoul Strackx. “Meltdown: reading kernel memory from user space”. In: *Communications of the ACM* 63.6 (21. Mai 2020), S. 46–56. ISSN: 0001-0782, 1557-7317. DOI: 10.1145/3357033.

[5] Wenjie Xiong und Jakub Szefer. “Survey of Transient Execution Attacks and Their Mitigations”. In: *ACM Computing Surveys* 54.3 (30. Apr. 2022), S. 1–36. ISSN: 0360-0300, 1557-7341. DOI: 10.1145/3442479.

[6] Jason Kim, Stephan Van Schaik, Daniel Genkin und Yuval Yarom. “iLeakage: Browser-based Timerless Speculative Execution Attacks on Apple Devices”. In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. CCS ’23: ACM SIGSAC Conference on Computer and Communications Security. Copenhagen Denmark: ACM, 15. Nov. 2023, S. 2038–2052. ISBN: 979-8-4007-0050-7. DOI: 10.1145/3576915.3616611.

2026-08-14

## Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

Literatur

Literatur

Literatur II

[4] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Jann Horn, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom, Mike Hamburg und Raoul Strackx. “Meltdown: reading kernel memory from user space”. In: *Communications of the ACM* 63.6 (21. Mai 2020), S. 46–56. ISSN: 0001-0782, 1557-7317. DOI: 10.1145/3357033.

[5] Wenjie Xiong und Jakub Szefer. “Survey of Transient Execution Attacks and Their Mitigations”. In: *ACM Computing Surveys* 54.3 (30. Apr. 2022), S. 1–36. ISSN: 0360-0300, 1557-7341. DOI: 10.1145/3442479.

[6] Jason Kim, Stephan Van Schaik, Daniel Genkin und Yuval Yarom. “iLeakage: Browser-based Timerless Speculative Execution Attacks on Apple Devices”. In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. CCS ’23: ACM SIGSAC Conference on Computer and Communications Security. Copenhagen Denmark: ACM, 15. Nov. 2023, S. 2038–2052. ISBN: 979-8-4007-0050-7. DOI: 10.1145/3576915.3616611.

[7] Jason Kim, Jalen Chuang, Daniel Genkin und Yuval Yarom. “FLOP: Breaking the Apple M3 CPU via False Load Output Predictions”. In: (2025).

[8] Jason Kim, Daniel Genkin und Yuval Yarom. “SLAP: Data Speculation Attacks via Load Address Prediction on Apple Silicon”. In: (2025).

[9] Mathe Hertogh, Dave Quakkelaar, Thijs Raymakers, Mahesh Hari Sarma, Marius Muench und Herbert Bos. “Rain: Transiently Leaking Data from Public Clouds Using Old Vulnerabilities”. In: *Vorveröffentlichung* ().

2026-08-14

Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

└ Literatur

└ Literatur

Literatur III

[7] Jason Kim, Jalen Chuang, Daniel Genkin und Yuval Yarom. “FLOP: Breaking the Apple M3 CPU via False Load Output Predictions”. In: (2025).

[8] Jason Kim, Daniel Genkin und Yuval Yarom. “SLAP: Data Speculation Attacks via Load Address Prediction on Apple Silicon”. In: (2025).

[9] Mathe Hertogh, Dave Quakkelaar, Thijs Raymakers, Mahesh Hari Sarma, Marius Muench und Herbert Bos. “Rain: Transiently Leaking Data from Public Clouds Using Old Vulnerabilities”. In: *Vorveröffentlichung* ().

# TEAs und CbCCs Ursache

- moderne CPUs sind hochkomplexe Maschinen
- mathematische Modelle der theoretischen Informatik werden simuliert
- tatsächliche Berechnungen werden durch abweichende Modelle optimiert
- Beispiele sind spekulative- und OoO-Execution, sowie Cache selbst
- Nutzen dieser Konzepte zum Ableiten unzugänglicher Informationen

2026-08-14

Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick  
└─Ergänzungen

TEAs und CbCCs Ursache

- TEAs und CbCCs Ursache
- moderne CPUs sind hochkomplexe Maschinen
  - mathematische Modelle der theoretischen Informatik werden simuliert
  - tatsächliche Berechnungen werden durch abweichende Modelle optimiert
  - Beispiele sind spekulative- und OoO-Execution, sowie Cache selbst
  - Nutzen dieser Konzepte zum Ableiten unzugänglicher Informationen

# TEAs und CbCCs Ursache

- moderne CPUs sind hochkomplexe Maschinen
- mathematische Modelle der theoretischen Informatik werden simuliert
- tatsächliche Berechnungen werden durch abweichende Modelle optimiert
- Beispiele sind spekulative- und OoO-Execution, sowie Cache selbst
- Nutzen dieser Konzepte zum Ableiten unzugänglicher Informationen

## Die Folgen

TEAs und CbCCs stellen genauso wie Side-Channel eine fundamentale Schwachstelle in modernen Computersystemen dar. Sie können vollständig ohne physischen Zugriff ausnutzt werden, obwohl Konzepte der Hardware die Ursache für die Schwachstellen sind.

2026-08-14

## Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

### Ergänzungen

TEAs und CbCCs Ursache

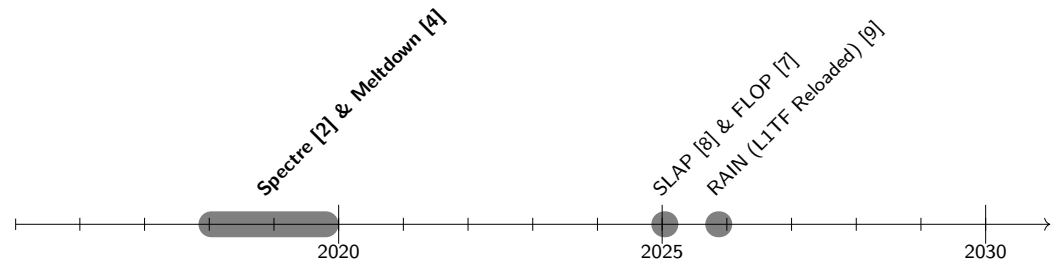
TEAs und CbCCs Ursache

- moderne CPUs sind hochkomplexe Maschinen
- mathematische Modelle der theoretischen Informatik werden simuliert
- tatsächliche Berechnungen werden durch abweichende Modelle optimiert
- Beispiele sind spekulative- und OoO-Execution, sowie Cache selbst
- Nutzen dieser Konzepte zum Ableiten unzugänglicher Informationen

#### Die Folgen

TEAs und CbCCs stellen genauso wie Side-Channel eine fundamentale Schwachstelle in modernen Computersystemen dar. Sie können vollständig ohne physischen Zugriff ausnutzt werden, obwohl Konzepte der Hardware die Ursache für die Schwachstellen sind.

# Verschiedene Angriffe



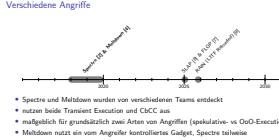
- Spectre und Meltdown wurden von verschiedenen Teams entdeckt
- nutzen beide Transient Execution und CbCC aus
- maßgeblich für grundsätzlich zwei Arten von Angriffen (spekulative- vs OoO-Execution)
- Meltdown nutzt ein vom Angreifer kontrolliertes Gadget, Spectre teilweise

2026-08-14 Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

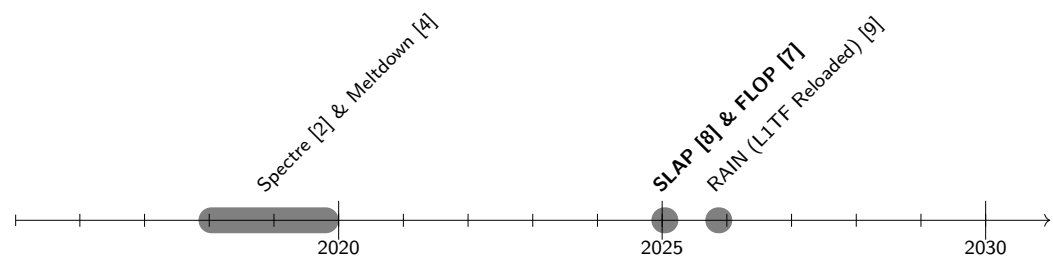
└─Ergänzungen

└─Verschiedene Angriffe

- Spectre & Meltdown Ende 2017 bis 2020 entdeckt und veröffentlicht- stellen neue Art von Angriff mit Transient Execution dar- Kategorisierung aller Angriffe in spekulative Ausführung oder OoO-Execution- Meltdown wird Code vollständig vom Angreifer kontrolliert, Spectre je nach Variante unterschiedlich



# Verschiedene Angriffe

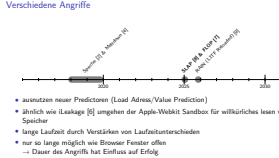


- ausnutzen neuer Predictoren (Load Adress/Value Prediction)
- ähnlich wie iLeakage [6] umgehen der Apple-Webkit Sandbox für willkürliches lesen von Speicher
- lange Laufzeit durch Verstärken von Laufzeitunterschieden
- nur so lange möglich wie Browser Fenster offen  
→ Dauer des Angriffs hat Einfluss auf Erfolg

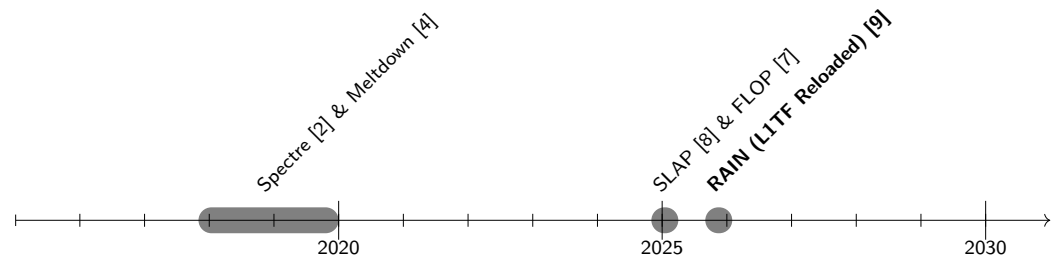
2026-08-14 Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick  
└Ergänzungen

└Verschiedene Angriffe

- LAP & LVP -¿ erster Nachweis + Exploit- zukünftig weitere Verbreitung solcher Predictoren?-"Vorgänger"Leakage (Spectre-Angriff mit anderem Predictor + Timing Amplification)- Lesen von Speicher innerhalb des Browser-Prozess -¿ SLAP und FLOP deutlich begrenzt, aber möglich-Laufzeit recht hoch, Nutzer darf Tab nicht schließen + Angreifer öffnet Ziel Webseite



# Verschiedene Angriffe

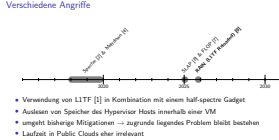


- Verwendung von L1TF [1] in Kombination mit einem half-spectre Gadget
- Auslesen von Speicher des Hypervisor Hosts innerhalb einer VM
- umgeht bisherige Mitigationen → zugrunde liegendes Problem bleibt bestehen
- Laufzeit in Public Clouds eher irrelevant

2026-08-14

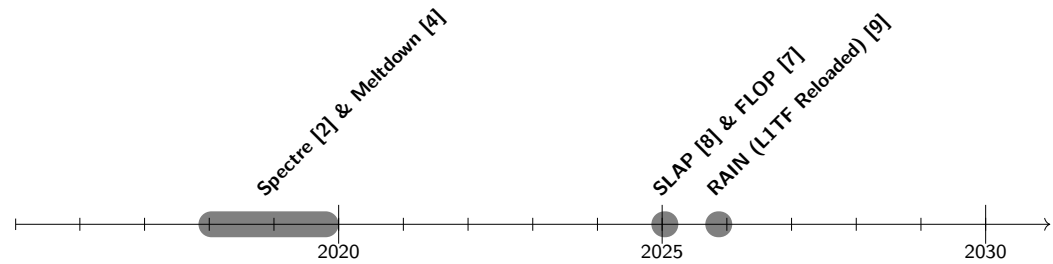
## Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

- └Ergänzungen
- └Verschiedene Angriffe



- L1TF Angriff von 2018 -> wurde „quasi“ Mitigiert- L1TF Reloaded nutzt half-spectre Gadget + L1TF- half-spectre = spekulatives Laden eines Secrets (out of bound), ohne Adressen abhängig von Wert zu laden- Gadget lädt beliebige Adresse (out of bound) in Cache -> lesen während Transient Execution mit L1TF- Konkreter Exploit: Auslesen von Hypervisor Memory in KVM (Public Cloud) -> Laufzeit weniger relevant, da unbegrenzt Zeit

# Verschiedene Angriffe



- Gadget für Encodierung in den CbCC häufig von Angreifer kontrolliert  
→ Encodierung und Technik kann frei gewählt werden
- Relevanz der Laufzeit für allgemeines Risiko solcher Angriffe
- teilweise relevant für Frage der Umsetzbarkeit

2026-08-14

Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

Ergänzungen

Verschiedene Angriffe

Verschiedene Angriffe

- Gadget für Encodierung in den CbCC häufig von Angreifer kontrolliert  
→ Encodierung und Technik kann frei gewählt werden
- Relevanz der Laufzeit für allgemeines Risiko solcher Angriffe
- teilweise relevant für Frage der Umsetzbarkeit

- viele Angriffe mit Angreifer kontrolliertem Gadget (alle hier gezeigten)- CbCC kann frei gewählt werden - neben Technik besonders Encodierung- Laufzeit für Risiko eines konkreten Angriffs relevant- Frage praktische Umsetzbarkeit bei sehr hoher Laufzeit



- Ziel ist grundsätzlicher Vergleich von bitweiser und Array-Index Encodierung
- Ausführung einzelner Übertragungen semi-zufälliger Eingabewerte
- unterschiedlicher Stride, Anzahl Bit, Trainingsläufe
- Speichern der Laufzeit, Eingabe, Ausgabe, Stride, Anzahl Bit, Trainingsläufe

2026-08-14

## Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

### └ Ergänzungen

### └ Messungen

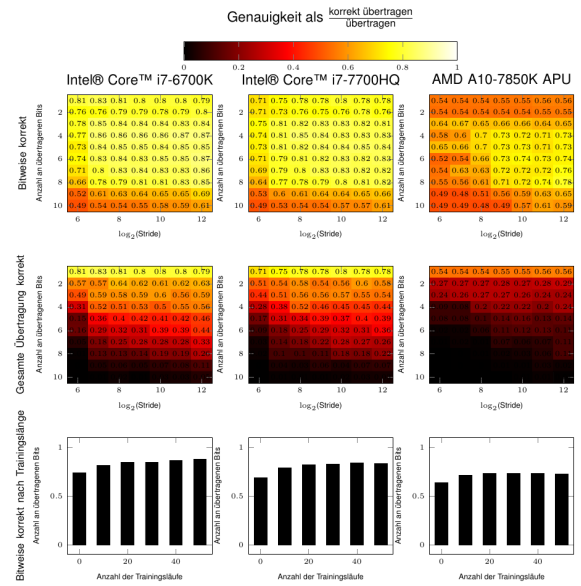
- Ziel an sich bitweise Encodierung zu demonstrieren und mit Array-Index zu vergleichen- entwickeltes Programm führt eine Übertragung von Eingabewert mit angegebener Encodierung durch- Eingabe semi zufälliger Werte zum übertragen (Randomisierung anhand von Index -i jedes Mal gleiche Werte)- Ausführung über 100 Iterationen je Parameter-Kombination- Parameter: Stride, Anzahl übertragener Bit, Anzahl Trainingsläufe

Messungen

- Ziel ist grundsätzlicher Vergleich von bitweiser und Array-Index Encodierung
- Ausführung einzelner Übertragungen semi-zufälliger Eingabewerte
- unterschiedlicher Stride, Anzahl Bit, Trainingsläufe
- Speichern der Laufzeit, Eingabe, Ausgabe, Stride, Anzahl Bit, Trainingsläufe

Auffälligkeiten bitweise

- steigende Genauigkeit bei unter 20 Trainingsläufen (im Gegensatz zu Array-Index)
- schon bei niedrigem Stride hohe Genauigkeit (ab etwa  $2^8$ )
- Genauigkeit gegenüber Array-Index ähnlich, aber grundsätzlich schlechter



2026-08-14

Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer Überblick

Ergänzungen

Messungen

Messungen

Genauigkeit als % korrekt übertragen

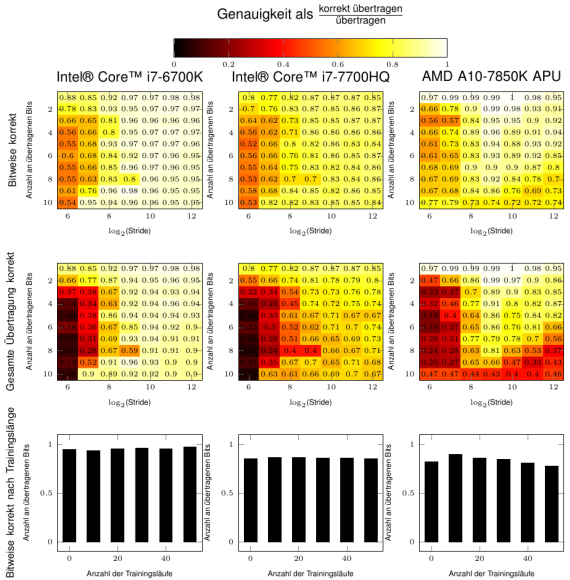
Intel Core™ i7-6700K, Intel Core™ i7-7700HQ, AMD A10-7850K APU

Auffälligkeiten bitweise

- steigende Genauigkeit bei unter 20 Trainingsläufen (im Gegensatz zu Array-Index)
- schon bei niedrigem Stride hohe Genauigkeit (ab etwa  $2^8$ )
- Genauigkeit gegenüber Array-Index ähnlich, aber grundsätzlich schlechter

- BITWEISE:- wie erwartet gute Genauigkeit bis Bitanzahl (Limit durch Anzahl Instruktionen)- Bis etwa 20 Trainingsläufe steigt Genauigkeit -¿ logisch wegen Predictor -¿ bei Array-Index nicht der Fall??- auch bei niedrigem Stride gute Genauigkeit, ab recht niedrigem Stride konstant- Übertragung wie bei Array-Index klar erkennbar, grundsätzlich schlechtere Genauigkeit

# Messungen



## Auff\u00e4lligkeiten Array-Index

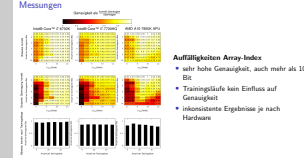
- sehr hohe Genauigkeit, auch mehr als 10 Bit
- Trainingsl\u00e4ufe kein Einfluss auf Genauigkeit
- inkonsistente Ergebnisse je nach Hardware

2026-08-14

Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution: ein technischer \u00dcberblick

Erg\u00e4nzungen

Messungen



- ARRAY-INDEX:- Sehr sehr hohe Genauigkeit mit deutlich \u00fcber 90% korrekten \u00dcbertragungen- wie erwartet keine Limitierung Bit Anzahl, Vergleich bitweise- hier keine konsistenter Einfluss von Trainingsl\u00e4ufen auf Genauigkeit erkennbar- Ergebnisse des AMD A10 Prozessors sehr auff\u00e4llig - 100% bei 1 bit, geringere Genauigkeit mit steigendem Stride, h\u00f6chste Genauigkeit bei geringerem Stride?? etc.